

```
[1] 0

$counts
function gradient
      32      NA

$convergence
[1] 0

$message
NULL

Mensajes de aviso perdidos
In optim(100, myCuadratica) :
  one-diml optimization by Nelder-Mead is unreliable:
use "Brent" or optimize() directly
```

Esta función, al buscar mínimos en funciones objetivos de cualquier tipo, no puede inferir a priori la estructura de la función a optimizar, por lo cual acepta como parámetros funciones cuya única variable sea un vector. O sea, permite camuflar cualquier número de variables independientes en un vector de variables independiente. En castellano, si queremos minimizar una función de dos variables, no podemos declararla como `function(x,y)`, sino que debemos declararla como `function(x)` donde "x" es un vector de dos componentes.

`optim()` aproxima el mínimo de la función que le pasemos como parámetro, pero funciona mediante métodos de búsqueda y, como tal, puede quedar trabada en un óptimo local. Nos pide que le pasemos "par"⁴⁵ para tener una idea desde donde arrancar a buscar. En este caso, "par" podría ser cualquier valor:

```
optim(25, myCuadratica)

$par
[1] 0

$value
[1] 0

$counts
function gradient
      32      NA

$convergence
[1] 0

$message
NULL

Mensajes de aviso perdidos
In optim(25, myCuadratica) :
  one-diml optimization by Nelder-Mead is unreliable:
```

⁴⁵ Cuidado, no confundir con la **función** `par()` utilizada para configurar gráficos.

```
use "Brent" or optimize() directly
```

```
optim(500000, myCuadratica)
```

```
$par  
[1] 0
```

```
$value  
[1] 0
```

```
$counts  
function gradient  
32 NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
Mensajes de aviso perdidos  
In optim(5e+05, myCuadratica) :  
one-diml optimization by Nelder-Mead is unreliable:  
use "Brent" or optimize() directly
```

```
optim(-500000, myCuadratica)
```

```
$par  
[1] 50000
```

```
$value  
[1] 2.5e+09
```

```
$counts  
function gradient  
10 NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
Mensajes de aviso perdidos  
In optim(-5e+05, myCuadratica) :  
one-diml optimization by Nelder-Mead is unreliable:  
use "Brent" or optimize() directly
```

¿Qué pasó?, funcionó bien para 25 y 50.000, pero no para -50.000. Lo que pasó es que el método por defecto de búsqueda de *optim()* no está preparado para búsquedas unidimensionales (fíjense que lo último que devuelve *optim* es un aviso acerca de la no confiabilidad del algoritmo "Nelder-Mead" para búsquedas unidimensionales, y nos sugiere usar el método "Brent"). Hagámosle caso:

```
optim(-500000, myCuadratica, method="Brent")
```

```
Error en optim(-5e+05, myCuadratica, method = "Brent") :  
'lower' and 'upper' must be finite values
```

¿Y ahora qué?. Bueno, "Brent" exige que le acotemos el espacio de búsqueda:

```
optim(-500000, myCuadratica, method="Brent", lower=-510000,  
upper=510000)
```

```
$par  
[1] 0  
  
$value  
[1] 0  
  
$counts  
function gradient  
NA NA  
  
$convergence  
[1] 0  
  
$message  
NULL
```

Ahora sí, probemos con diferentes valores iniciales y cotas:

```
optim(-23.8, myCuadratica, method="Brent", lower=-100,  
upper=100)$value  
  
optim(-230000000.8, myCuadratica, method="Brent", lower=-  
100000, upper=100000)$value  
  
optim(-23.8, myCuadratica, method="Brent", lower=-1000000,  
upper=100)$value  
  
optim(-11111111, myCuadratica, method="Brent", lower=-  
1000000, upper=1000000)$value
```

Vemos que aproxima bien la el mínimo de esta función cuadrática. Tiene un error de precisión mínimo cuando las cotas del espacio de búsqueda son muy grandes en valor absoluto, a efectos prácticos es bastante insignificante. También vemos que, como todo objeto, podemos acceder a sus miembros. Veamos un poco su estructura:

```
optimizacion <- optim(-23.8, myCuadratica, method="Brent",  
                      lower=-100000, upper=100000)  
  
str(optimizacion)  
  
optimizacion$par # Valores de las variables de decisión
```

```
optimizacion$value # Valor de la función objetivo
optimizacion$convergence # Vale 0 si la solución converge
optimizacion$message # Nos devuelve los mensajes de alerta
```

Si lo pensamos un poco, seguimos con el mismo problema que vimos cuando explicamos el uso de funciones, no podemos pasarle parámetros adicionales a nuestra función. Definamos una cuadrática en su forma general:

```
myCuadraticaGeneral <- function(x, coefa, coefb, coefc){
  coefa*(x^2) + coefb*x + coefc
}
```

Supongamos que quiero minimizar la cuadrática con coeficientes 0.5, -25, -10, respectivamente. Podría crear una nueva función proxy, que solo tome como parámetro la variable de decisión, y encapsule la asignación de los coeficientes de la función cuadrática:

```
myCuadraticaParticular <- function(x){
  myCuadraticaGeneral(x, 0.5, -25, -10)
}

optim(0, myCuadraticaParticular)
```

Pero si tengo que optimizar muchas cosas voy a tener un montón de funciones cargadas en la memoria. ¿Por qué no usar funciones anónimas?:

```
optim(0, (function(x){myCuadraticaGeneral(x,0.5,-25,-10)}))
```

¡¡¡Chan!!!, fíjense lo que hicimos: creamos una versión de la función *myCuadraticaParticular()* al vuelo, declarándola dentro de *optim()*, con toda esta declaración entre paréntesis. Esta función existe solo dentro de la ejecución de *optim()*, después desaparece de memoria. Si se fijan, es la misma declaración que *myCuadráticaParticular()* pero sin asignación a ninguna variable (en realidad la estamos asignando al parámetro "fn" de *optim()*). Lo bueno es que pasamos los parámetros dentro de la misma *optim()*, por lo que no tengo que andar creando nuevas funciones proxys si quiero optimizar cuadráticas con distintos coeficientes..

Bueno, pero optimizar funciones de una sola variable es aburrido. Hagamos algo más complicado:

```
myFuncionNoLineal <- function(x,y){  
  x^2 + y^2  
}
```

Tenemos ahora una función que toma como entrada un vector de dos componentes, y devuelve como resultado una combinación no lineal de las mismas.

Visualicémosla con *persp()*:

```
?persp
```

persp() necesita como input un vector de puntos para el eje X, uno para el eje Y, y una matriz en el cual el elemento [x,y] sea el valor de la función para (x,y):

```
x <- seq(0:20)-10  
y<-x  
valoresFuncion <- outer(x,y,myFuncionNoLineal)  
persp(x,y,valoresFuncion, col="royalblue", phi=30, theta=30)
```

optim() necesita que la función acepte un solo vector de variables independientes, así que nuestra definición de *myFuncionNoLineal()* con dos parámetros no está en el formato que necesitamos.

Para optimizarlo, utilicemos una función anónima que se encargue de la conversión:

```
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}))
```

El método de resolución para problemas por defecto funciona bien con funciones de dos variables.

Este método es el "Nelder – Mead"⁴⁶, el cual optimiza basado solo en los valores de la función objetivo. El método funciona bastante bien pero es lento. Igualmente, al no necesitar gradientes para operar, trabaja muy bien frente a funciones no diferenciables.

⁴⁶ Publicado en 1965

Un método basado en gradientes es el "BFGS", el cual es un método del tipo "cuasi-Newton":

```
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}),method="BFGS")
```

Comparemos el atributo "counts" de optim usando "Nelder - Mead" versus "BFGS":

```
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}))  
    )$counts  
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}),  
    method="BFGS")$counts
```

Este atributo tiene dos componentes. El primero (function) nos indica la cantidad de iteraciones del procedimiento, el segundo la cantidad de veces que se debió recalcular el gradiente (en este caso, solo aplica a "BFGS"). A simple vista, "BFGS" es más "rápido", o sea, necesitó menos iteraciones para llegar al objetivo (y tuvo mayor grado de precisión). El método "BFGS" necesita de una función para computar el gradiente. Si no la pasamos, calcula el gradiente por diferencias finitas, pero podemos pasarle la función generadora de gradientes:

```
gradienteMyFuncionNoLineal <- function(x){  
  c(2*x[1], 2*x[2])  
}
```

Como es una función generadora de gradientes, me tiene que devolver un vector de la misma cantidad de componentes que el vector de variables independientes de la función a optimizar. Entonces:

```
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}),  
    gr= gradienteMyFuncionNoLineal, method="BFGS")
```

Y vemos que, al no tener que aproximar por diferencias finitas, optim trabaja muchísimo más rápido (4 iteraciones contra 19 en mi computadora)

optim() acepta muchos más parámetros adicionales, en general pasados como lista en el parámetro "control":

```
optim(c(100,100), (function(x){myFuncionNoLineal(x[1],
x[2])})),
      gr= gradienteMyFuncionNoLineal, method="BFGS",
      control=list(trace=1))
```

"control=list(trace=1)" nos devuelve los valores iniciales y finales de la función objetivo, por ejemplo.

Hasta ahora minimizamos solamente, ¿cómo hacemos para maximizar?. Definamos una función más complicada:

```
funcionMasComplicada <- function(x,y){
  r <- sqrt(x^2+y^2)
  10 * sin(r)/r
}
```

Hagamos su gráfico:

```
x <- seq(-15, 15, length= 40)
y <- x
matrizGraficacion <- outer(x, y, funcionMasComplicada)
matrizGraficacion[is.na(matrizGraficacion)] <- 1
persp(x, y, matrizGraficacion, theta = 30, phi = 30,
      expand = 0.5, col = "royalblue", ltheta = 120,
      shade = 0.75, ticktype = "detailed")
```

Y maximicémoslas con el argumento de control "fnscale":

```
optim(c(1,10), (function(x){funcionMasComplicada(x[1],
x[2])})),
      control=list(trace=6, fnscale=-1))
```

Con un método basado en búsqueda, esta función es candidata a quedar atrapada en un óptimo local. Con "trace=6" vimos la secuencia de pasos que hizo para encontrar el valor óptimo. Probemos con "BFGS":

```
optim(c(1,10), (function(x){funcionMasComplicada(x[1],
x[2])}),
      method="BFGS", control=list(trace=6, fnscale=-1))
```

¿Y si quisiéramos optimizar una función más genérica, es decir, que admita que le modifiquemos sus parámetros?. Armemos un paraboloide:

```
myParaboloideElptico <- function(x,y,par_a,par_b,par_corr_x,
                                par_corr_y, par_termIndep){
  ((x+par_corr_x)/par_a)^2 + ((y+par_corr_y)/par_b)^2 +
  par_termIndep
}
```

Para optimizarla podemos usar una función implícita. Supongamos que grabamos los valores de los parámetros en variables:

```
a <- 2
b <- 3
corr_x <- 1
corr_y <- 0
termIndep <- -10
optim(c(1,10), (function(x){myParaboloideElptico(x[1],
x[2],
a,b,corr_x,corr_y,
termIndep)}),
      method="BFGS", control=list(trace=6))
```

7.2. Optimizando con restricciones

Hasta ahora, entonces, estuvimos trabajando con *optim()* para optimización no lineal. Es una función muy buena, pero trabaja minimizando funciones no restringidas. En la práctica de la IO, en general nos tocan problemas que se caracterizan por un conjunto de restricciones que limitan nuestro rango de soluciones. En el caso que queramos minimizar funciones no lineales sujetas a restricciones lineales, disponemos de la función "*constrOptim()*". En líneas generales, es muy

parecida a "*optim()*" pero se le agregan parámetros relativos a las restricciones. Minimicemos un paraboloide elíptico, sujeto a restricciones del tipo $\geq$. Para ello, usemos la función que creamos anteriormente:

```
myParaboloideEliptico <- function(x,y,par_a,par_b,par_corr_x,
                                par_corr_y, par_termIndep){
  ((x+par_corr_x)/par_a)^2 + ((y+par_corr_y)/par_b)^2 +
  par_termIndep
}
```

Ahora defino las restricciones:

```
matrizA <- matrix(c(1,0,0,1), nrow=2, byrow=TRUE)
vectorb <- c(5,10)
```

Y optimizo:

```
constrOptim(c(100,100),
function(x){myParaboloideEliptico(x[1],x[2],1,1,0,0,0)},
          grad=NULL, ui=matrizA,ci=vectorb)
```

Al igual de lo que pasaba con *optim()*, debo proporcionarle un valor inicial de búsqueda, que en este caso, además, debe estar dentro de la región factible.

El objeto que devuelve es similar al que devuelve *optim()*, pero con los parámetros adicionales "outer.iteration" y "barrier.value", donde el primero son las llamadas que se hacen a *optim()* y el segundo el valor de la barrera.

7.3. Armando nuestra propia versión de OPTIM

Ahora creemos nuestra propia versión de *optim()* personalizada. ¿Qué personalización le queremos dar?, bueno, a priori, hagamos que si mi variable de decisión es unidimensional, ejecute el método Brent por defecto:

Veamos primero la definición de *optim()*:

optim

Hagamos ahora nuestro prototipo:

```
myOptim <- function (valIniciales, funcion, gradiente =
NULL, ..., cotaInf = -Inf,
                    cotaSup = Inf, listaControl = list(), hessiano =
FALSE) {
}

```

El valor inicial se pasa en el parámetro `valInicial`, así que miremos cuantas componentes tiene y, mediante la función `if()`, determinemos si usamos Brent o no:

```
myOptim <- function (valIniciales, funcion, gradiente =
NULL, ..., cotaInf = -Inf, cotaSup = Inf, listaControl =
list(), hessiano = FALSE) {

```

```
  # Chequeo que método usar
  if (length(valIniciales) == 1){
    metodo <- "Brent"
  }
  else {
    metodo <- "Nelder-Mead"
  }

```

```
#Llamo a optim():
  print(paste("El método utilizado fue: ", metodo))
  optim(valIniciales, funcion, gradiente, ..., method =
metodo, lower=cotaInf,
        upper=cotaSup, control=listaControl,
hessian=hessiano)
}

```

Probémoslo:

```
myCuadratica <- function(x){
  x^2 + 3
}

```

```
myOptim(c(10), myCuadratica)
```

¿Qué pasó? Bueno, recordemos que el método Brent requiere que las cotas máximas y mínimas sean finitas. Por lo tanto, modifiquemos nuestra definición:

```
myOptim <- function (valIniciales, funcion, gradiente =
NULL, ..., cotaInf = -Inf, cotaSup = Inf, listaControl =
list(), hessiano = FALSE) {

# Chequeo que método usar
if (length(valIniciales) == 1){
  metodo <- "Brent"
  if (cotaInf == -Inf){
    cotaInf <- -1000000000000000
  }
  if (cotaSup == Inf){
    cotaSup <- 1000000000000000
  }
}
else {
  metodo <- "Nelder-Mead"
}

# Llamo a optim():
print(paste("El método utilizado fue: ", metodo))
optim(valIniciales, funcion, gradiente, ..., method=metodo
, lower = cotaInf,
      upper = cotaSup, control = listaControl, hessian =
hessiano)
}
```

Llamemos a nuestra versión mejorada:

```
myOptim(c(10), myCuadratica)
```

Ojo, esta función sirve en el caso unidimensional cuando los valores posibles están dentro de las cotas por defecto (-1000000000000000 y 1000000000000000), sino debemos explicitarlas.

Puedo hacer ahora que, en caso de optimización multidimensional, si le paso el gradiente resuelva por BFGS:

```
myOptim <- function (valIniciales, funcion, gradiente =
NULL, ..., cotaInf = -Inf, cotaSup = Inf, listaControl =
list(), hessiano = FALSE) {

  # Chequeo que método usar
  if (length(valIniciales) == 1){
    metodo <- "Brent"
    if (cotaInf == -Inf){
      cotaInf <- -1000000000000000
    }
    if (cotaSup == Inf){
      cotaSup <- 1000000000000000
    }
  }
  else {
    if (is.null(gradiente)){
      metodo <- "Nelder-Mead"
    }
    else {
      metodo <- "BFGS"
    }
  }

  # Llamo a optim():
  print(paste("El método utilizado fue: ", metodo))
  optim(valIniciales, funcion, gradiente, ..., method=metodo
, lower = cotaInf,
      upper = cotaSup, control = listaControl, hessian =
hessiano)
}
```


Llamemos ahora a *myOptim()* con y sin gradiente:

```
myFuncionNoLineal <- function(x,y){  
  x^2 + y^2  
}  
  
gradienteMyFuncionNoLineal <- function(x){  
  c(2*x[1], 2*x[2])  
}  
  
myOptim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])})))  
  
myOptim(c(100,100), (function(x){myFuncionNoLineal(x[1],  
x[2])}), gradienteMyFuncionNoLineal)
```

Algoritmos genéticos

8.1. Algoritmos genéticos

Hasta ahora, trabajamos con problemas lineales resolviéndolos mediante el método simplex (usando tres funciones diferentes), y con problemas no lineales utilizando tres métodos diferentes con la función *optim()* (Brent, Nelder-Mead y BFGS), con búsqueda por gradientes o búsquedas aleatorias. Vamos a estudiar ahora como implementar en R un conjunto de métodos más robustos englobados dentro de la familia de los algoritmos evolutivos, los algoritmos genéticos.

Como siempre, para empezar a trabajar, definamos nuestro directorio de trabajo y borremos el contenido del workspace:

```
setwd("D://Proyectos//IOR//scripts")
rm(list=ls())
```

El paquete que vamos a utilizar es el “genalg”

```
install.packages("genalg")
library("genalg")
```

Parte del poder de los algoritmos genéticos es que su operación abstraer cualquier detalle de la estructura del problema, reduciéndose todas las variables de decisión a un "cromosoma". ⁴⁷La forma estándar de trabajo es mediante cromosomas con "genes" binarios. Así que empecemos con este tipo de problemas.

Supongamos el siguiente problema de minimización:

```
# Max Z = XA * 24 + XB * 20
#
# Sujeto a:
#
# 10 * XA + 7 * XB <= 200
# 1 * XA + 0.5 * XB <= 12
# 1 * XA + 0 * XB <= 15
# 0 * XA + 1 * XB <= 24
# 1 * XA + 1 * XB <= 20
```

⁴⁷ Raro pasar de hablar de conceptos como solución factible a conceptos como cromosomas, ¿no?

Con todas las variables no negativas y donde el problema consiste en maximizar la cantidad de vapor generada por tonelada de carbón (X) en una caldera, siendo la primera restricción el presupuesto que se dispone para gastar en carbón, la segunda la cantidad máxima de kg de humo por tonelada de carbón quemado que se puede generar, la segunda y la tercera las capacidades de proceso para los carbones A y B, y la última la capacidad de transporte de carbón.

A fin de utilizar el paquete "genalg", es necesario convertir el problema anterior al formalismo de los algoritmos genéticos. Lo primero es decidir cómo codificamos las variables. A priori, dado que tenemos dos variables, nuestros cromosomas deberían tener dos genes solamente:

CROMOSOMA = GEN_A | GEN_B

El siguiente paso es definir con cuantos bits codificamos cada gen. En general, si bien se pueden aplicar algoritmos genéticos a problemas no acotados, siempre es preferible acotar, aunque sea artificialmente, todas las variables de decisión. En este caso, tenemos dos restricciones que podemos usar como cotas naturales:

$$\begin{aligned} 1 * X_A + 0 * X_B &\leq 15 \quad - \text{Cota } X_A \\ 0 * X_A + 1 * X_B &\leq 24 \quad - \text{Cota } X_B \end{aligned}$$

X_A debe pertenecer al intervalo $[0; 16]$. Como vamos a codificarla por bits, que solo admiten dos valores posibles (0 y 1), debemos buscar la potencia de dos igual o inmediatamente mayor a 16. $16 = 2$ elevado a la 4^{ta} potencia, por lo cual con 4 bits podemos codificar X_A :

0000 = 0
...
1111 = 15

Haciendo el mismo análisis con X_B , podemos ver que la potencia de 2 inmediatamente superior a 24 es $32 = 2^5$, por lo cual:

00000 = 0
...
11111 = 31

Si lo pensamos un poco, trabajar en binario puede ser un poco tedioso, así que hagamos dos funciones para hacer las conversiones:⁴⁸

```

pasarABaseBinariaDesdeDecimal <- function(numero){
  numero.redondeado <- floor(numero)
  binario <- c()
  while (numero.redondeado >= 2){
    binario <- c(binario, numero.redondeado %% 2)
    numero.redondeado <- numero.redondeado %% 2
  }
  binario <- c(binario, numero.redondeado)
  binario[length(binario):1]
}

pasarABaseDecimalDesdeBinaria <- function(arrayDeBits){
  cantidadBits <- length(arrayDeBits)
  multiplicador <- rep(2, times=cantidadBits)
  multiplicador <- multiplicador^(order(multiplicador)-1)
  multiplicador <- multiplicador[order(multiplicador,
decreasing = TRUE)]
  sum(arrayDeBits * multiplicador)
}

pasarABaseBinariaDesdeDecimal(10)
[1] 1 0 1 0

pasarABaseDecimalDesdeBinaria(c(1,0,1,0))
[1] 10

```

Ya codificada nuestra solución, tenemos que analizar cómo armar nuestra función objetivo. A lo que van a decir, "la función objetivo es $X_A * 24 + X_B * 20$ ", lo cual es cierto. Pero los algoritmos genéticos no trabajan con restricciones más allá de las cotas de las variables, por lo cual es necesario incluir las restricciones a la función objetivo en transformándola en una otra función. Una forma bastante sencilla puede ser que la función objetivo sea $X_A * 24 + X_B * 20$ si no se viola ninguna restricción, y si se violan, ya que es un problema de maximización, reducir el valor objetivo en "M" (valor absoluto del desvío), siendo "M" un número muy grande. Entonces, podemos escribir nuestra función objetivo como:

```

funobj <- function(XA, XB){
  # Defino el valor de M
  M <- 50

```

⁴⁸ Como ya sabemos, mucho mejor si las guardamos en un script para poder llamarlas cuando necesitamos.

```

# Chequeo las restricciones:
desvio <- abs(min(0,200-(10 * XA + 7 * XB)))
desvio <- desvio + abs(min(0,12-(1 * XA + 0.5 * XB)))
desvio <- desvio + abs(min(0,16-(1 * XA + 0 * XB))) # No
necesaria, ya está acotada
desvio <- desvio + abs(min(0,24-(0 * XA + 1 * XB))) # Es
necesaria, la cota calculada es 32
desvio <- desvio + abs(min(0,20-(1 * XA + 1 * XB)))

XA * 24 + XB * 20 - M * desvio
}

```

Pero recordemos que tenemos que trabajar con cadenas de bits, así que hagamos una función objetivo que acepte una solución potencial al estilo de los algoritmos genéticos, y que internamente llame a nuestra función de evaluación:

```

funobjbin <- function(cromosoma){
  cromosoma.XA <- cromosoma[1:4]
  cromosoma.XB <- cromosoma[5:9]

  XA <- pasarABaseDecimalDesdeBinaria(cromosoma.XA)
  XB <- pasarABaseDecimalDesdeBinaria(cromosoma.XB)

  funobj(XA, XB)
}

```

Codificadas nuestras variables de decisión y armada la función objetivo, ya podemos usar el paquete "genalg". La función a utilizar es *rbga.bin()*, la cual minimiza, por lo tanto tenemos que invertir el signo de nuestra función objetivo. Es este caso, mediante una función anónima:⁴⁹

```

myProb <- rbga.bin(size=9, evalFunc = function(x){-
1*funobjbin(x)}), showSettings = TRUE)

```

```

GA Settings
Type           =
Population size = 200
Number of Generations = 100
Elitism        = 40
Mutation Chance = 0.1

```

```

Search Domain
Var 1 = [,]
Var 0 = [,]

```

Noten que demora cierto tiempo en terminar el análisis. Veamos los resultados:

⁴⁹ Por razones de espacio, los vectores "population", "evaluations" y "mean" no se muestran en su totalidad en este libro.

```
myProb
```

```
$type  
[1] "binary chromosome"
```

```
$size  
[1] 9
```

```
$popSize  
[1] 200
```

```
$iters  
[1] 100
```

```
$suggestions  
NULL
```

```
$population  
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]  
[1,] 0     1     0     0     1     0     0     0     0  
[2,] 0     1     0     0     1     0     0     0     0  
[3,] 0     1     0     0     1     0     0     0     0  
[4,] 0     1     0     0     1     0     0     0     0  
[5,] 0     1     0     0     1     0     0     0     0  
[6,] 0     1     0     0     1     0     0     0     0  
[7,] 0     1     0     0     1     0     0     0     0  
[8,] 0     1     0     0     1     0     0     0     0  
...
```

```
$elitism  
[1] 40
```

```
$mutationChance  
[1] 0.1
```

```
$evaluations  
[1] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[15] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[29] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[43] -416 -416 -96 -196 -416 -416 -361 -416 -416 -96 -416 -416 -  
416 -416  
...
```

```
$best  
[1] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[15] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[29] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[43] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[57] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[71] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[85] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[99] -416 -416
```

```
$mean  
[1] -142.230 -207.135 -261.915 -275.175 -329.960 -344.005 -353.000  
[8] -353.885 -352.705 -367.890 -378.365 -365.255 -373.680 -354.060  
[15] -337.930 -360.325 -390.730 -311.695 -369.665 -322.950 -366.875  
[22] -377.155 -362.000 -378.525 -367.885 -370.135 -371.840 -363.525  
[29] -366.775 -356.055 -381.345 -384.490 -373.285 -347.235 -365.435
```

...

```
attr(,"class")  
[1] "rbga"
```

Donde "size" es el tamaño de cada cromosoma. Usamos el "showSetting = TRUE" para mostrar la configuración por defecto que utilizamos. Analicemos un poco el objeto que nos devuelve la función:

```
myProb$popSize  
[1] 200
```

El algoritmo funcionó con un tamaño de población igual a 200, o sea, en cada iteración se evaluaron 200 potenciales soluciones del problema (en términos de Algoritmos Genéticos, cromosomas). Si vemos las soluciones:⁵⁰

```
myProb$population  
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]  
[1,] 0 1 0 0 1 0 0 0 0  
[2,] 0 1 0 0 1 0 0 0 0  
[3,] 0 1 0 0 1 0 0 0 0  
[4,] 0 1 0 0 1 0 0 0 0  
[5,] 0 1 0 0 1 0 0 0 0  
[6,] 0 1 0 0 1 0 0 0 0  
[7,] 0 1 0 0 1 0 0 0 0  
[8,] 0 1 0 0 1 0 0 0 0
```

Tenemos una matriz de 200 filas (una por cada cromosoma en la población con la mejor solución alcanzada) y 10 columnas (el tamaño de cada cromosoma que la función supone que estamos utilizando). Para obtener nuestra mejor solución:

```
myProb$evaluations  
[1] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416  
-416  
[15] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[29] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -  
416 -416  
[43] -416 -416 -96 -196 -416 -416 -361 -416 -416 -96 -416 -416 -  
416 -416  
[57] -416 -416 -416 -416 -320 -196 -416 -416 -320 -416 -320 -416 -  
416 -416  
[71] -320 -416 -416 -416 -416 -96 -416 -306 -320 -416 -416 -416 -  
416 -416  
[85] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 -96 -416 -  
416 -416  
[99] -416 -416 -96 -320 -416 -416 -416 -416 -416 -196 -416 -416 -  
416 -416
```

⁵⁰ Nuevamente, no mostramos las 200 soluciones, solo las 8 primeras.

```

[113] -416 -96 -416 -416 -416 -416 -416 -416 -96 -320 -416 -416 -
416 -320
[127] -416 -416 -416 -416 -416 -416 -416 -416 -416 -416 0 -416 -
416 -416
[141] -416 -416 -416 -416 -320 -416 -416 -416 -416 -416 -416 -306 -
416 -96
[155] -416 -416 -416 -416 -416 -416 -136 -320 -416 -96 -416 -416 -
416 -416
[169] -320 -416 -416 -416 -416 -320 -416 -416 -320 -96 -96 -416 -
416 -320
[183] -416 -361 -416 -416 -340 -416 -416 -416 -96 -416 -96 -96 -
96 -416
[197] -416 -196 -416 -416

```

Y tenemos los valores de función objetivo para cada uno de los 200 cromosomas en la población final. Nosotros nos tenemos que quedar con la solución con el mejor valor de función objetivo, entonces si con orden podemos obtener los índices ordenados:

```
order(myProb$evaluations)
```

```

[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
17
[18] 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
34
[35] 35 36 37 38 39 40 41 42 43 44 47 48 50 51 53 54
55
[52] 56 57 58 59 60 63 64 66 68 69 70 72 73 74 75 77
80
[69] 81 82 83 84 85 86 87 88 89 90 91 92 93 94 96 97
98
[86] 99 100 103 104 105 106 107 109 110 111 112 113 115 116 117 118
119
[103] 120 123 124 125 127 128 129 130 131 132 133 134 135 136 138
139 140
[120] 141 142 143 144 146 147 148 149 150 151 153 155 156 157 158
159 160
[137] 163 165 166 167 168 170 171 172 173 175 176 180 181 183 185
186 188
[154] 189 190 192 196 197 199 200 49 184 187 61 65 67 71 79
102 122
[171] 126 145 162 169 174 177 182 78 152 46 62 108 198 161 45
52 76
[188] 95 101 114 121 154 164 178 179 191 193 194 195 137

```

Podemos hacer:

```

myProb.solucion <-
myProb$population[order(myProb$evaluations)[1],]
myProb.solucion
[1] 0 1 0 0 1 0 0 0 0

```

Si quiero saber a qué valores de mis variables objetivos corresponden, las decodifico:

```

myProb.XA <-
pasarABaseDecimalDesdeBinaria(myProb.solucion[1:4])
myProb.XB <-
pasarABaseDecimalDesdeBinaria(myProb.solucion[5:9])

```


Y ya que estamos:

```
myProb.obj <- -1 * min(myProb$evaluations)
```

Entonces:

```
myProb.XA  
[1] 4
```

```
myProb.XB  
[1] 16
```

```
myProb.obj  
[1] 416
```

rbga.bin() trabajó con casi todos los parámetros por defecto, pero podemos modificarlos antes de hacer otra corrida:

```
myProb <- rbga.bin(size=9, popSize = 100, iters = 500,  
                  mutationChance = 0.05, elitism = 10,  
                  evalFunc = function(x){-1*funobjbin(x)},  
                  showSettings = TRUE)
```

```
GA Settings  
Type                =  
Population size     = 100  
Number of Generations = 500  
Elitism             = 10  
Mutation Chance     = 0.05
```

```
Search Domain  
Var 1 = [,]  
Var 0 = [,]
```

```
myProb.solucion <-  
myProb$population[order(myProb$evaluations)[1],]  
myProb.XA <-  
pasarABaseDecimalDesdeBinaria(myProb.solucion[1:4])  
myProb.XB <-  
pasarABaseDecimalDesdeBinaria(myProb.solucion[5:9])  
myProb.obj <- -1 * min(myProb$evaluations)
```

```
myProb.XA  
[1] 4
```

```
myProb.XB  
[1] 16
```

```
myProb.obj  
[1] 416
```

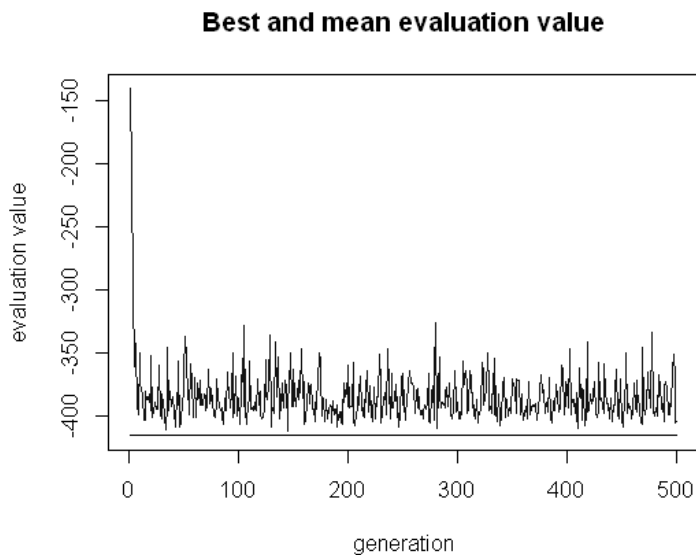
La función *rbga.bin()* es muy interesante para analizar cómo evoluciona la población en cada iteración. Analizando el objeto que devuelve, tenemos los atributos “mean” y “best”, que nos muestran para cada

iteración el promedio del valor objetivo y el mejor valor objetivo respecto de la población vigente en ese momento:

```
myProb$mean  
myProb$best
```

Podemos ver la evolución de las soluciones en un gráfico:

```
plot(myProb)
```



8.2. *Armando nuestras propias funciones*

El paquete "genalg" es bastante útil para la aplicación práctica de los Algoritmos Genéticos, pero quizás nos interese armar nuestra propia biblioteca de funciones para Algoritmos Genéticos, de manera de probar otras configuraciones o nuevas técnicas. Así que armemos una función básica de optimización basada en este paradigma para tener de ejemplo⁵¹. Primero que todo, armemos una función que nos genere una población inicial aleatoria:

```
generarPoblacionInicial <- function(tamanoCromosoma,  
tamanoPoblacion){
```

⁵¹ Nuevamente, recordemos de guardar estas funciones en un script independiente.

```

matrix(round(runif(tamanoCromosoma * tamanoPoblacion)),
       nrow = tamanoPoblacion,
       ncol = tamanoCromosoma,
       byrow = TRUE)
}

```

Vamos a probarla :

```
generarPoblacionInicial(5,10)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	0	0	0	0
[2,]	0	1	0	1	1
[3,]	0	0	0	1	0
[4,]	0	1	0	0	1
[5,]	1	0	1	1	1
[6,]	0	0	1	0	1
[7,]	1	1	1	0	1
[8,]	0	0	0	0	1
[9,]	1	1	1	1	0
[10,]	1	1	1	1	1

Hagamos una función para seleccionar las “n” mejores:

```

seleccionarMejores <- function (poblacion, funcionObjetivo,
cantMejores, minimizar = TRUE){
  valoresObj <- c()
  for (i in 1:nrow(poblacion)){
    valoresObj <- c(valoresObj,
funcionObjetivo(poblacion[i,1:ncol(poblacion)]))
  }
  order(valoresObj, decreasing = !minimizar)[1:cantMejores]
}

```

Observemos que hay que pasarle un parámetro denominado “funciónObjetivo”, que es la función de evaluación que utilizará para seleccionar las soluciones. En este caso, le pasamos una función en forma implícita que lo que hace es sumar las componentes del vector a evaluar. Probémosla:

```

datos <- generarPoblacionInicial(5,10)
datos

```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	0	1	1	0
[2,]	1	1	1	1	1
[3,]	0	1	1	1	1
[4,]	0	1	1	0	0
[5,]	0	0	1	1	0
[6,]	1	0	1	1	1
[7,]	0	0	0	1	0
[8,]	0	1	1	0	0
[9,]	1	1	1	0	1
[10,]	1	0	0	0	0

```
seleccionarMejores(datos, function(x){sum(x)}, cantMejores =
3, minimizar = FALSE)
[1] 2 3 6
```

Ya sabemos cómo generar la población inicial y como seleccionar por las mejores. Veamos cómo mutar a población:

```
mutarCromosomas <- function(poblacion, porcentajeMutacion){
  cantCromosomas <- nrow(poblacion)
  cromosomasMutados <- ceiling(poblacion *
porcentajeMutacion)
  cromosomasSeleccionados <-
sample(1:cantCromosomas,cromosomasMutados,replace=FALSE)
  bitsSeleccionados <-
sample(1:ncol(poblacion),cromosomasMutados,replace=TRUE)

  nuevaPoblacion <- poblacion
  nuevaPoblacion[cromosomasMutados,bitsSeleccionados] <-
  ((poblacion[cromosomasMutados,bitsSeleccionados]+1) %%
2)

  nuevaPoblacion
}
```

Probemos nuestra función de mutación:

```
datos
  [,1] [,2] [,3] [,4] [,5]
[1,]  1    0    1    1    0
[2,]  1    1    1    1    1
[3,]  0    1    1    1    1
[4,]  0    1    1    0    0
[5,]  0    0    1    1    0
[6,]  1    0    1    1    1
[7,]  0    0    0    1    0
[8,]  0    1    1    0    0
[9,]  1    1    1    0    1
[10,] 1    0    0    0    0

mutarCromosomas(datos, 0.25)
  [,1] [,2] [,3] [,4] [,5]
[1,]  1    1    1    1    0
[2,]  1    1    1    1    1
[3,]  0    1    1    1    1
[4,]  0    1    1    0    0
[5,]  0    0    1    1    0
[6,]  1    0    1    1    1
[7,]  0    0    0    1    0
[8,]  0    1    1    0    0
[9,]  1    1    1    0    1
[10,] 1    0    0    0    0
```

Y ahora armemos la función para generar hijos:

```
generarHijosDeMadreYPadre <- function(madre, padre,
cantHerenciaMadre){
  hijo <- padre
```

```

herenciaMadre <- sample(1:length(madre),
cantHerenciaMadre, replace = FALSE)
hijo[herenciaMadre] <- madre[herenciaMadre]
hijo
}

```

El parámetro “cantHerenciaMadre” define cuantos genes provienen de la madre, el resto son los genes del padre. La probamos:

```

generarHijosDeMadreYPadre(c(1,1,1,1,1), c(0,0,0,0,0), 3)
[1] 1 0 1 0 1

generarHijosDeMadreYPadre(c(1,1,1,1,1), c(0,0,0,0,0), 3)
[1] 0 1 1 1 0

generarHijosDeMadreYPadre(c(1,1,1,1,1), c(0,0,0,0,0), 3)
[1] 0 0 1 1 1

```

Como vemos, definimos cuantos genes hereda de la madre, pero la selección de dichos genes se produce aleatoriamente, a fin de poder generar hijos distintos de una misma pareja.

Tenemos que armar ahora una gran función que tome como parámetros todos los parámetros necesarios para ejecutar todas las funciones definidas en la biblioteca recién cargada. Su prototipo sería:

```

myAlgoritmoGenetico <- function(tamanoCromosoma,
tamanoPoblacion,cantMejores,porcentajeMutacion,cantHerenciaM
adre, nroIteraciones = 100, funcionObjetivo, minimizar =
TRUE){
}

```

Como vemos, tomamos todos los parámetros de todas las funciones, excepto aquellos que se pueden obtener de resultados de otras. Empecemos a armarla:

```

myAlgoritmoGenetico <- function(tamanoCromosoma,
tamanoPoblacion,cantMejores,porcentajeMutacion,cantHerenciaM
adre, nroIteraciones = 100, funcionObjetivo, minimizar =
TRUE){

  poblacion <- generarPoblacionInicial(tamanoCromosoma,
tamanoPoblacion)
  mejoresCromosomas <- c()
}

```

Para empezar, lo básico. En base al tamaño del cromosoma y el tamaño de la población deseada, generamos nuestra población inicial. Ya que

estamos, también creamos una variable del tipo vector, vacía por ahora, que se llama "mejoresCromosomas", y que nos servirá para almacenar los mejores individuos de la solución. Sigamos:

```
myAlgoritmoGenetico <- function(tamanoCromosoma,
tamanoPoblacion,cantMejores,porcentajeMutacion,cantHerenciaM
adre, nroIteraciones = 100, funcionObjetivo, minimizar =
TRUE){

  poblacion <- generarPoblacionInicial(tamanoCromosoma,
tamanoPoblacion)
  mejoresCromosomas <- c()

  for (i in 1:nroIteraciones){
    mejoresCromosomas <-
seleccionarMejores(poblacion,funcionObjetivo,cantMejores,
minimizar)
    nuevaPoblacion <- poblacion[mejoresCromosomas,]
  }
}
```

Agregamos ahora un bucle que se repite tantas veces como iteraciones hayamos pasado como parámetro. En cada iteración, lo primero que hacemos es seleccionar los mejores individuos de la población y los copiamos a una nueva variable. Recuerden que "poblacion" es una matriz, "mejoresCromosomas" un vector y "nuevaPoblacion" es entonces una matriz que proviene de filtrar a la "poblacion" actual por las filas asociadas a los mejores individuos de dicha población. Si estoy en la primer iteración, "poblacion" almacena a mi población actual, en iteraciones posteriores almacenamos a las nuevas poblaciones que vayamos creando.

Sigamos:

```
myAlgoritmoGenetico <- function(tamanoCromosoma,
tamanoPoblacion,cantMejores,porcentajeMutacion,cantHerenciaM
adre, nroIteraciones = 100, funcionObjetivo, minimizar =
TRUE){

  poblacion <- generarPoblacionInicial(tamanoCromosoma,
tamanoPoblacion)
  mejoresCromosomas <- c()

  for (i in 1:nroIteraciones){
    mejoresCromosomas <-
seleccionarMejores(poblacion,funcionObjetivo, cantMejores,
minimizar)
    nuevaPoblacion <- poblacion[mejoresCromosomas,]
    for (j in 1:(nrow(poblacion) -
length(mejoresCromosomas))){
```

```

        padres <- sample(mejoresCromosomas, 2, replace =
FALSE)
        nuevaPoblacion <- rbind(nuevaPoblacion,
generarHijosDeMadreYPadre (poblacion[padres[1],], poblacion[pa
dres[2],], cantHerenciaMadre))
    }
}
}

```

Dentro del bucle principal (el asociado a las iteraciones) creo un nuevo bucle encargado de generar una nueva población en base a la selección de los mejores individuos. Para ello, repito el bucle la cantidad de veces que hagan falta para llenar la matriz "nuevaPoblacion" con el mismo tamaño que la población original, pero generando cada elemento mediante el cruzamiento de las mejores soluciones. Por tanto, selecciono los padres aleatoriamente en cada iteración, los cruzo y los agrego a la nueva población. Noten que la aleatoriedad para generar hijos es alta, ya que los padres se eligen al azar, y los hijos se generan seleccionando cromosomas al azar de sus padres.

Sigamos:

```

myAlgoritmoGenetico <- function(tamanoCromosoma,
tamanoPoblacion, cantMejores, porcentajeMutacion, cantHerenciaM
adre, nroIteraciones = 100, funcionObjetivo, minimizar =
TRUE) {

    poblacion <- generarPoblacionInicial(tamanoCromosoma,
tamanoPoblacion)
    mejoresCromosomas <- c()

    for (i in 1:nroIteraciones){
        mejoresCromosomas <-
seleccionarMejores(poblacion, funcionObjetivo, cantMejores,
minimizar)
        nuevaPoblacion <- poblacion[mejoresCromosomas,]
        for (j in 1:(nrow(poblacion) -
length(mejoresCromosomas))){
            padres <- sample(mejoresCromosomas, 2, replace =
FALSE)
            nuevaPoblacion <- rbind(nuevaPoblacion,
generarHijosDeMadreYPadre (poblacion[padres[1],], poblacion[pa
dres[2],], cantHerenciaMadre))
        }
        poblacion <- mutarCromosomas(nuevaPoblacion,
porcentajeMutacion)
    }
    poblacion
}

```

Por último, me aseguro de mutar un poco la población al final de cada iteración del bucle principal, y devuelvo la última población generada al finalizar dicho bucle.

Probémoslo a ver que devuelve. Si maximizamos, como la función objetivo es la suma de sus componentes, la población devuelta debería tener mayoría de vectores con todos sus componentes valiendo 1. Para el caso de minimización, la mayoría de los vectores debería tener sus componentes valiendo 0.⁵²

```
myAlgoritmoGenetico(9,100,10, 0.1,5,100,function(x){sum(x)},
minimizar=FALSE)
```

```
[1,] [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
[2,] 1 1 1 1 1 1 1 1 1 1
[3,] 1 1 1 1 1 1 1 1 1 1
[4,] 1 1 1 1 1 1 1 1 1 1
...
```

```
myAlgoritmoGenetico(9,100,10, 0.1,5,100,function(x){sum(x)},
minimizar=TRUE)
```

```
[1,] [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
[2,] 0 0 0 0 0 0 0 0 0 0
[3,] 0 0 0 0 0 0 0 0 0 0
[4,] 0 0 0 0 0 0 0 0 0 0
[5,] 0 0 0 0 0 0 0 0 0 0
...
```

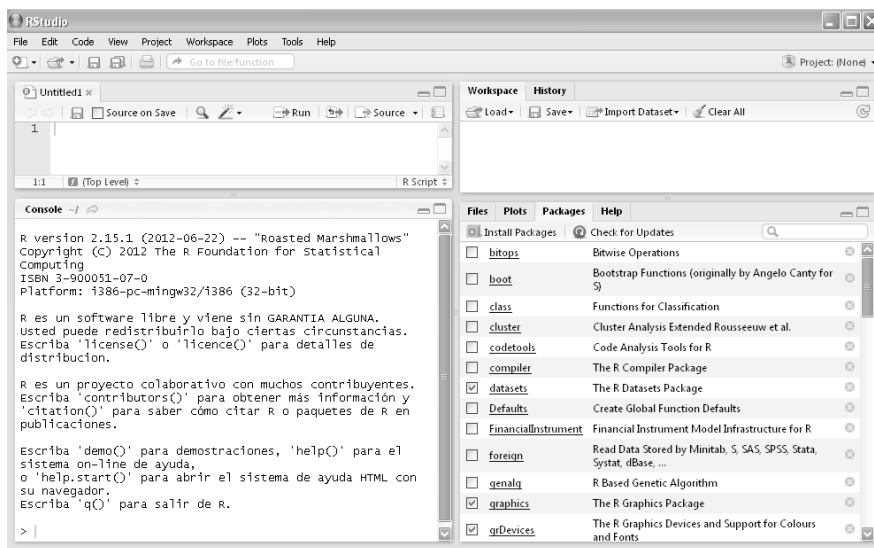
⁵² Nuevamente, no se muestran los resultados completos, por cuestiones de espacio.

Anexo I: Interacción con RStudio

¿Por qué utilizar RStudio?

Si bien la distribución estándar de R se puede utilizar sin problemas, las IDE basadas en ella nos ofrecen algunas facilidades extras para el trabajo diario. Una de las distribuciones más populares es RStudio.

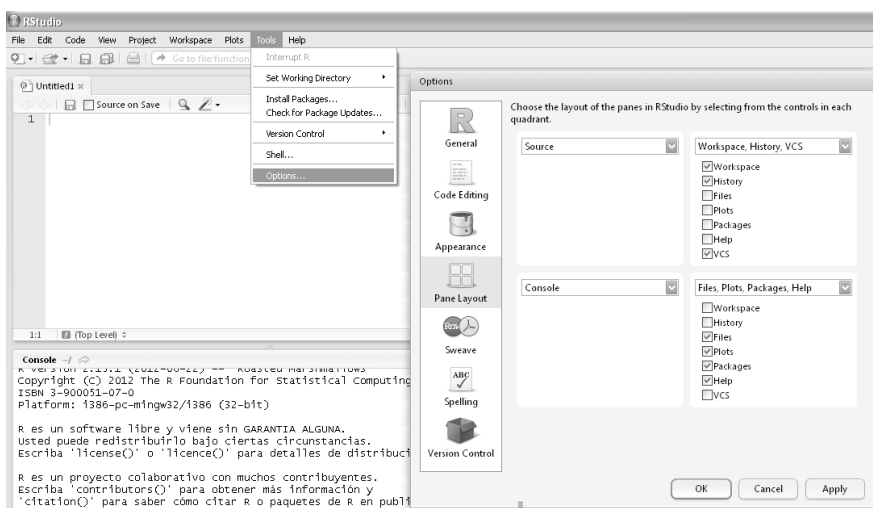
RStudio nos presenta una interfaz clara que, además de la consola, nos permite monitorear y acceder a varias estructuras del sistema.



En la configuración que nosotros utilizamos, podemos obtener, adicionalmente al acceso a la consola (panel inferior izquierdo), acceso al listado de paquetes instalados, con la posibilidad de cargarlos en memoria y consultar su ayuda con un clic del mouse (panel inferior derecho), un detalle de todos los objetos cargados en memoria (panel superior derecho) y un editor de script para no tener que salir de R cuando queremos guardar nuestras funciones a archivos (panel superior izquierdo). Pero mejor, vayamos paso a paso.

Configurando el layout de visualización

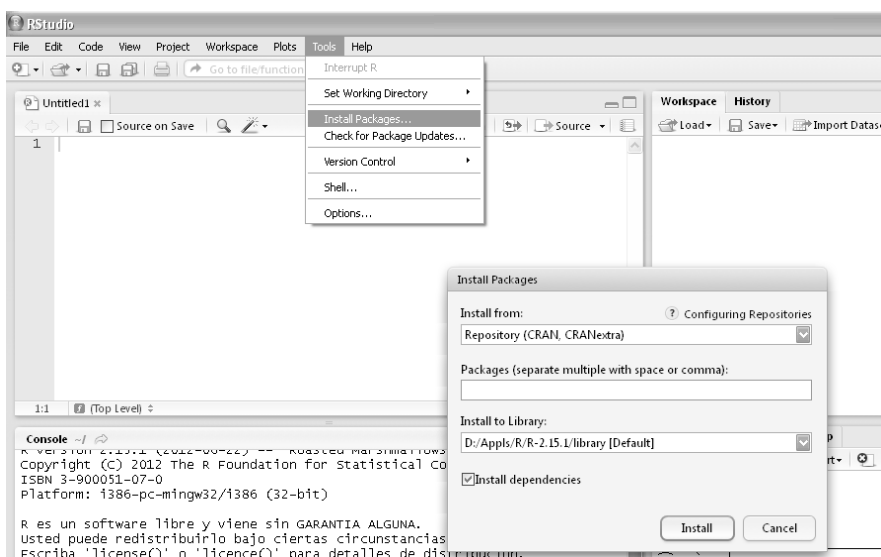
La interfaz gráfica de RStudio es altamente personalizable. Desde el menú “Tools”, mediante el submenú “Options” podemos acceder a la ventana de configuración, dentro de la cual encontramos la solapa “Panel Layout”:



En esta ventana podemos definir el contenido de cada una de las cuatro secciones del layout, seleccionando las opciones los desplegables de cada cuadro y tildando o destildando las pestañas que queremos ver.

Trabajando con paquetes

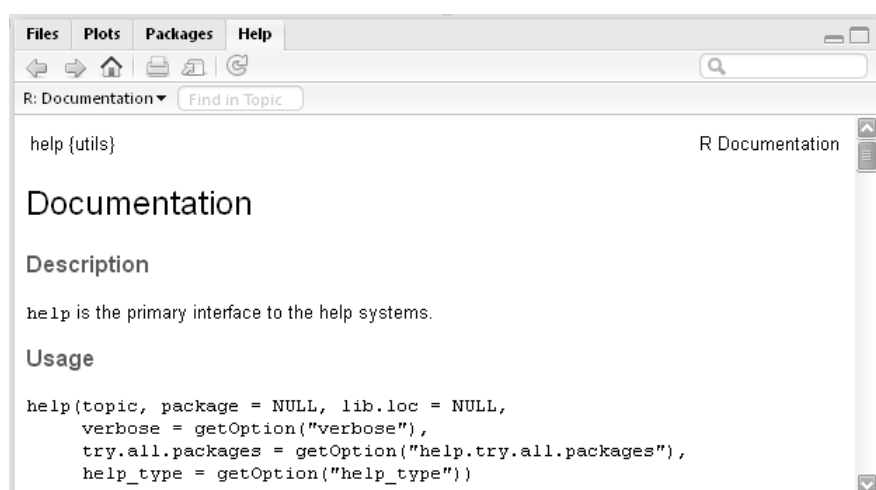
La gestión de paquetes se puede hacer directamente desde la interfaz gráfica. Desde **“Tool/Install Packages...”** podemos sustituir al comando `install.packages()`:



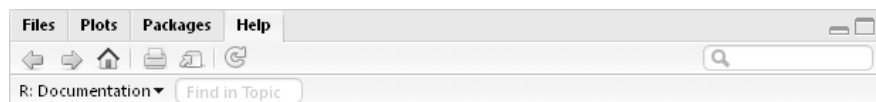
Desde el cuadro de diálogo emergente seleccionamos el repositorio que queremos utilizar, tipeamos los paquetes a instalar y definimos en cual biblioteca queremos instalarlo.

Consultando la ayuda

Desde la consola podemos acceder a los comandos habituales de ayuda, como *help()*. Sin embargo, a diferencia del entorno estándar de R la ayuda se abre en uno de los paneles del entorno, en vez de en una ventana adicional:

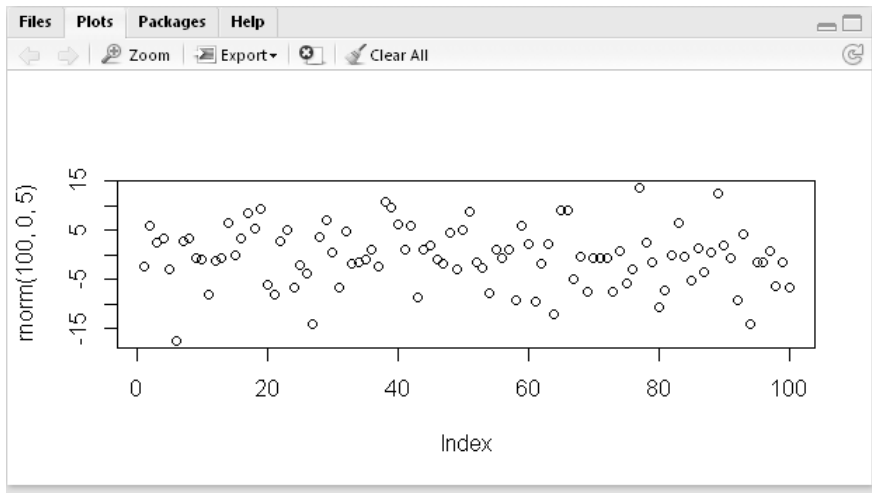


Se puede acceder en forma directa desde la pestaña “**Help**”. Además. El panel ofrece una barra de navegación, con comandos para ir atrás, adelante, a la portada de la ayuda, imprimirla, mostrarla en una nueva ventana (en el navegador de internet activo), refrescar el tema y buscar algún tópico (a la derecha de la barra, equivale a usar *help.search()*):



Visualizando gráficos

Dentro de los paneles tenemos la opción de visualización de gráficos:



RStudio lleva un historial de gráficos dibujados, por lo tanto, podemos acceder a los mismos sin necesidad de volver a tipear los comandos con los cuales lo generamos.

Dentro de este panel, podemos ver los gráficos anteriores y posteriores que estén grabados en el historial, hacerles zoom (en una ventana externa), grabarlos a un archivo, eliminar el gráfico del historial o borrar todo el historial.

Monitoreando la memoria de trabajo

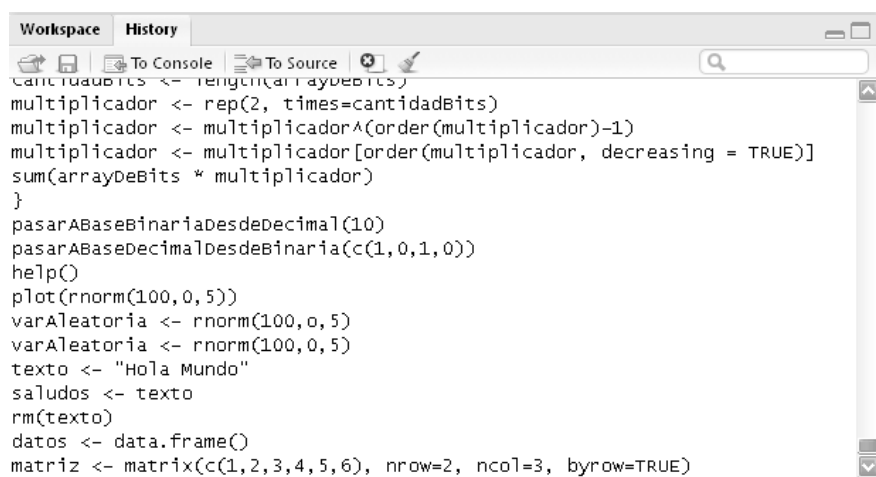
Una forma más rápida de revisar el contenido de la memoria de trabajo, sin necesidad de llamar al comando `ls()`, es visualizar el panel “Workspace”:

Workspace		History
Load		Save
Import Dataset		Clear All
Data		
datos	0 obs. of 0 variables	
matriz	2x3 double matrix	
Values		
saludos	"Hola Mundo"	
varAleatoria	numeric[100]	

Desde el panel podemos abrir un workspace existente, grabarlo en memoria, cargar un conjunto de datos desde archivo (siendo equivalente a, entre otros, hacer un `read.csv()`) y limpiar la memoria de trabajo (como si hiciéramos `rm(list=ls())`). Además, haciendo clic sobre alguno de los datos almacenados en la memoria, llamamos al comando `fix()`.

Revisando el historial

Otra muy buena funcionalidad es poder revisar el historial de comandos ingresados a la consola desde la pestaña **“History”**:



```
Workspace History
[Icons: Save, Load, To Console, To Source, Refresh, Find]
cantidadBits <- length(arrayDeBits)
multiplicador <- rep(2, times=cantidadBits)
multiplicador <- multiplicador^(order(multiplicador)-1)
multiplicador <- multiplicador[order(multiplicador, decreasing = TRUE)]
sum(arrayDeBits * multiplicador)
}
pasarABaseBinariaDesdeDecimal(10)
pasarABaseDecimalDesdeBinaria(c(1,0,1,0))
help()
plot(rnorm(100,0,5))
varAleatoria <- rnorm(100,0,5)
varAleatoria <- rnorm(100,0,5)
texto <- "Hola Mundo"
saludos <- texto
rm(texto)
datos <- data.frame()
matriz <- matrix(c(1,2,3,4,5,6), nrow=2, ncol=3, byrow=TRUE)
```

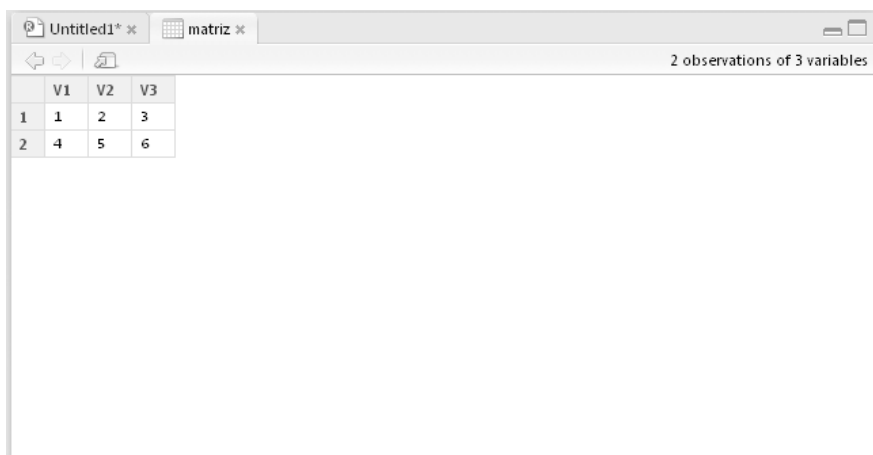
Escribiendo nuestros script

Para escribir nuestros propios script no es necesario utilizar un editor externo. Tenemos el panel **“Source”** para ello:

Desde el mismo podemos generar guardar nuestros scripts, disponemos de algunas funcionalidades de edición básicas (como por ejemplo, buscar y reemplazar) y tenemos la posibilidad de mandar los sentencias escritas en el script a la consola, o bien hacer un `source()` y ejecutar todo el script. Todos los archivos que creamos desde **“File/New”** o que abramos desde **“File/Open”** se abrirán en este panel. Además, desde el mismo, tenemos la posibilidad de compilar lo escrito como HTML, a fin de generar un archivo de documentación, por ejemplo.

Ejecutando View()

Por último, en RStudio, cada vez que llamemos a la `View()` no se nos abre una nueva ventana, sino que se nos muestran los datos en una nueva pestaña del panel “**Source**”:



The screenshot shows the RStudio interface with the Source panel active. It displays a data frame with 2 observations and 3 variables. The data is presented in a table format with columns V1, V2, and V3, and rows 1 and 2.

	V1	V2	V3
1	1	2	3
2	4	5	6

Anexo II: Más paquetes para optimización

Como buscar más paquetes de optimización

Las posibilidades del uso de R para optimización no se limitan a lo visto aquí. Existen numerosos paquetes registrados en CRAN para optimización. Uno puede consultarlo en la “Task View” dedicada al tema “Optimization and Mathematical Programming”⁵³. No podemos tratar todos, ni tampoco darles toda la profundidad que se merecen pero, igualmente, a continuación reseñamos algunos de estos paquetes.

ROI

Este paquete presenta un marco de trabajo para modelar problemas de programación lineal y no lineal independientemente del solver a utilizar.

optimx

Paquete que permite encapsular la llamada a diferentes métodos de optimización para problemas no lineales dentro de una única función (*optimx*). Basicamente es una extensión de la función *optim*, pero que puede llamar a muchos más algoritmos de resolución.

dfoptim

Paquete para optimizaciones no lineales mediante métodos de búsquedas sin necesidad de declarar gradientes. Implementa algoritmos de Nelder-Mead y Hooke-Jeeves.

hydroPSO

Poderosísimo paquete que implementa algoritmos de Optimización de Enjambre de partículas.

⁵³ Al día de la publicación de este libro, el enlace a dicha Task View es:
<http://cran.r-project.org/web/views/Optimization.html>

rgenoud

Paquete que implementa la función *rgenoud*, la cual permite resolver problemas de optimización mediante la aplicación de una combinación de algoritmos evolutivos con métodos basados en derivadas.

DEoptim

Paquete para resolución de problemas mediante técnicas de optimización basadas en algoritmos de evolución diferencial.

Bibliografía

- [1]. R Core Team (2012). R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. ISBN 3-900051-07-0, URL <http://www.R-project.org/>.
- [2]. R Development Core Team. 2000. Introducción a R, V1.0.1.
- [3]. A. J. Arriaza Gómez y otros. 2008. Estadística Básica con R y RCommander. Ediciones UCA
- [4]. R CRAN. Mathemtical Programming Task View.
- [5]. Norman Matloff. From Algorithms to Z-Scores: Probabilistic and Statistical Modeling in Computer Science.
<http://heather.cs.ucdavis.edu/matlo/probstatbook.html>
- [6]. Norman Matloff. R for Programmers.
<http://heather.cs.ucdavis.edu/~matloff/132/NSPpart.pdf>
- [7]. Paula Elosua. Introducción al entorno R. ISBN: 978-84-9860-497-9
- [8]. Andrew Robinson. Icebreaker. <http://cran.r-project.org/doc/contrib/Robinson-icebreaker.pdf>
- [9]. Sean Luke. Essentials of Metaheuristics.
<http://cs.gmu.edu/~sean/book/metaheuristics/>
- [10]. Jason Brownlee. Clever Algorithms.
<http://www.cleveralgorithms.com/>
- [11]. R-Enthusiasts: <http://gallery.r-enthusiasts.com>

